

ReduLink: Authenticated Redundancy-Suppressed Transmission for Effective Reconstructed Throughput over Encrypted WANs

Michél Nguyen Independent Researcher | minh.systems Draft version 0.9 submission-ready evidence, artifact, and protocol-scope revision, June 2026

Manuscript type: Protocol design, systems modeling, and security analysis

Abstract. Ethernet and wide-area transport protocols are typically evaluated by physical or nominal line rate, yet many real traffic classes contain repeated objects, pages, layers, templates, or versioned artifacts. ReduLink is a representation-layer model and candidate Deduplex-QUIC integration profile for authenticated reference substitution between cooperative encrypted endpoints. Rather than increasing physical line rate, ReduLink can increase effective reconstructed payload throughput when a receiver validates compact references against an epoch-scoped dictionary. This draft specifies the ReduLink abstraction, candidate QUIC integration semantics, sender and receiver state machines, security properties, privacy modes, and conservative accounting rules. The accompanying artifact validates byte-exact reconstruction, fail-closed reference handling, public-fixture benchmarks, deterministic target-class fixtures, a fixed-block reuse approximation baseline, local wall-clock/RSS cost columns, and a TCP endpoint-reconstruction prototype. The artifact is not a QUIC implementation and does not validate production cryptography, replay windows, congestion fairness, 0-RTT, migration, or cross-tenant privacy enforcement.

Keywords: network redundancy elimination, QUIC, Ethernet, WAN optimization, content-defined chunking, authenticated references, effective throughput, congestion control, privacy-preserving deduplication

1. Introduction

The apparent capacity of a network link is usually expressed as a physical or negotiated line rate. A 1 Gbit/s Ethernet connection can place roughly one billion bits per second on the wire, and a 10 Gbit/s connection can place ten times as many. Modern Ethernet standards already extend far beyond these rates, including active IEEE 802.3 work on 200 Gbit/s, 400 Gbit/s, 800 Gbit/s, and 1.6 Tbit/s Ethernet. Therefore, a credible research contribution should not claim that a software protocol creates a physically faster Ethernet link.

This paper instead asks a different question: can a protocol increase the amount of original application payload reconstructed per second without changing the physical link rate? For workloads with repeated content, the answer is yes in principle. If a sender transmits a 64-byte authenticated reference instead of an 8 KiB repeated chunk, the receiver can reconstruct more payload than was placed on the wire. The link has not become physically faster. Rather, the wire now carries semantic references to previously authenticated bytes.

Network redundancy elimination and WAN optimization are established areas. Prior work showed that repeated byte strings occur in network traffic and that redundancy elimination can save substantial bandwidth in enterprise settings. However, the Internet has changed. Traffic is now predominantly encrypted, QUIC and TLS protect payloads from in-network modification, and middleboxes cannot safely inspect or rewrite encrypted byte streams. The opportunity is therefore not transparent network compression. The opportunity is endpoint-negotiated redundancy suppression that composes with encryption, congestion control, privacy, and loss recovery.

ReduLink is proposed as such a method. It can be instantiated as a QUIC extension, a VPN mode, a CDN-to-client feature, a data-center transport shim, or a SmartNIC-assisted link-layer service in trusted environments. Its central mechanism is simple: full chunks populate an epoch-scoped receiver dictionary, while later repetitions are encoded as

authenticated reference frames. If a reference is missing, unverifiable, stale, or unsafe, the sender falls back to ordinary full-chunk transmission.

1.1 Research question

The research question is: How can a transport or link-adjacent protocol increase effective reconstructed payload throughput for redundant traffic while preserving end-to-end correctness, transport security, congestion fairness, and privacy?

1.2 Contributions

- A protocol abstraction for authenticated redundancy-suppressed transmission over encrypted WANs.
- An epoch-scoped dictionary model that avoids unsafe global cross-user deduplication.
- A candidate Deduplex-QUIC integration profile with FULL, REF, MISS, and DICT_ACK behavior, plus negotiated transport parameters for dictionary scope, expansion bounds, 0-RTT policy, and reference policy.
- A deduplication-aware congestion accounting rule that counts wire bytes against congestion but reconstructs expanded payload bytes at the application boundary.
- A conservative throughput model and CSV-backed evidence matrix showing both useful and weak target classes.
- A reproducible artifact package with fixed and content-defined chunking, deterministic target-class fixtures, public-corpora fetch scripts, fixed-block reuse baseline, local wall-clock/RSS cost columns, socket prototype, automated tests, CI, CSV outputs, and generated figures.
- A threat model and privacy analysis covering reference forgery, replay, dictionary poisoning, expansion abuse, and content-existence side channels.
- An evaluation plan for software, VPN, CDN, backup, container, and SmartNIC deployments.

2. Background and related work

2.1 Ethernet line rate versus effective payload throughput

Ethernet standards define physical and link-layer capabilities. A redundancy-suppressed protocol cannot make a 1 Gbit/s physical interface emit 2 Gbit/s of electrical or optical signal. It can only reduce the number of wire bytes needed to deliver a given original byte sequence. The distinction is essential. ReduLink is a method for effective bandwidth expansion, not physical-rate expansion.

2.2 Redundancy elimination

Spring and Wetherall introduced protocol-independent redundancy elimination by identifying repeated byte strings in network traffic and replacing redundant transfers with compact encodings. Later enterprise trace studies found substantial redundancy, but also showed that the value depends on the workload and deployment location. Anand et al. reported that a large share of middlebox bandwidth savings in enterprise traces came from redundancy within each client's own traffic, which motivates end-system designs.

2.3 End-system redundancy elimination

EndRE proposed redundancy elimination as an end-system service rather than a middlebox function. This direction is particularly relevant to modern encrypted traffic because endpoint-controlled redundancy suppression can happen before encryption or inside a trusted transport endpoint. ReduLink differs by focusing explicitly on a modern encrypted transport instantiation, authenticated references, epoch dictionaries, reference-miss recovery, and congestion semantics.

2.4 Content-defined chunking

Content-defined chunking divides byte streams into variable-sized chunks using rolling fingerprints. It is widely used in deduplication systems because insertion or deletion near the beginning of a file does not necessarily shift all later chunk boundaries. For ReduLink, content-defined chunking is useful because repeated content may not align with packet boundaries. However, chunking must be fast enough for line-rate operation and parameterized safely to avoid side channels.

2.5 QUIC and encrypted transports

QUIC provides stream multiplexing over UDP and integrates TLS security. Because QUIC encrypts payload data and protects much control information, in-network transformation of QUIC traffic is generally incompatible with the protocol design. A ReduLink mode for the public Internet must therefore be negotiated and executed by endpoints, such as clients, servers, VPN clients, CDN edges, or application gateways.

2.6 Gap in the literature

Existing work demonstrates that redundancy elimination can save bandwidth. The unresolved modern problem is how to express redundancy suppression as a secure, encrypted-WAN-compatible protocol primitive with explicit reconstruction invariants, bounded expansion, per-connection privacy, loss recovery, and congestion accounting. ReduLink addresses this gap.

Table 1a. Related-work comparison and ReduLink gap.

System	Placement	E2E encrypted traffic	Auth refs / epochs	QUIC semantics	Privacy posture
Spring/Wetherall RE	Middlebox/network	Weak for QUIC/TLS	No / no	No	Network-visible payloads
LBFS / rsync	File/application	Yes, file-specific	Partial / no	No	Application-specific
EndRE	Endpoint service	Better than middleboxes	Limited / unclear	No	Enterprise endpoint scope
SmartRE	Coordinated middleboxes	Weak for E2E QUIC	No / no	No	Enterprise/provider scope
ReduLink	Endpoint/QUIC/VPN	Yes, endpoint-negotiated	Yes / yes	Specified	Per-connection default; no global cross-user mode

3. Design goals and non-goals

3.1 Design goals

Goal	Meaning
Correctness	The reconstructed byte stream must equal the original sender byte stream.
Security	References must be authenticated and bound to the connection, epoch, and chunk identity.
Privacy	The protocol must not reveal global content possession across unrelated users or origins.
Deployability	The protocol must work as an endpoint feature, QUIC extension, VPN mode, or trusted SmartNIC feature.
Graceful fallback	The protocol must revert to ordinary transmission on reference miss, dictionary mismatch, loss, or negotiation failure.
Fairness	Congestion control must remain fair to other traffic by accounting for actual wire bytes.
Bounded expansion	The receiver must cap reconstructed bytes per reference and per epoch to avoid expansion abuse.

Table 1. ReduLink design goals.

3.2 Non-goals

- ReduLink does not increase physical Ethernet signaling rate.

- ReduLink does not require transparent middleboxes to inspect encrypted traffic.
- ReduLink does not promise gains for compressed video, encrypted random-like data, or already optimized media streams.
- ReduLink does not use a global cross-user deduplication dictionary in public WAN mode.
- ReduLink is not a replacement for compression. It is a reference-substitution protocol for repeated content.

4. ReduLink protocol overview

ReduLink operates between two cooperating endpoints. The sender and receiver first negotiate support. Once enabled, the sender maintains a bounded dictionary of chunks that the receiver is believed to know. The receiver maintains a corresponding dictionary of chunks it has authenticated and stored. Dictionary membership is scoped by an epoch, which limits lifetime, memory use, and replay exposure.

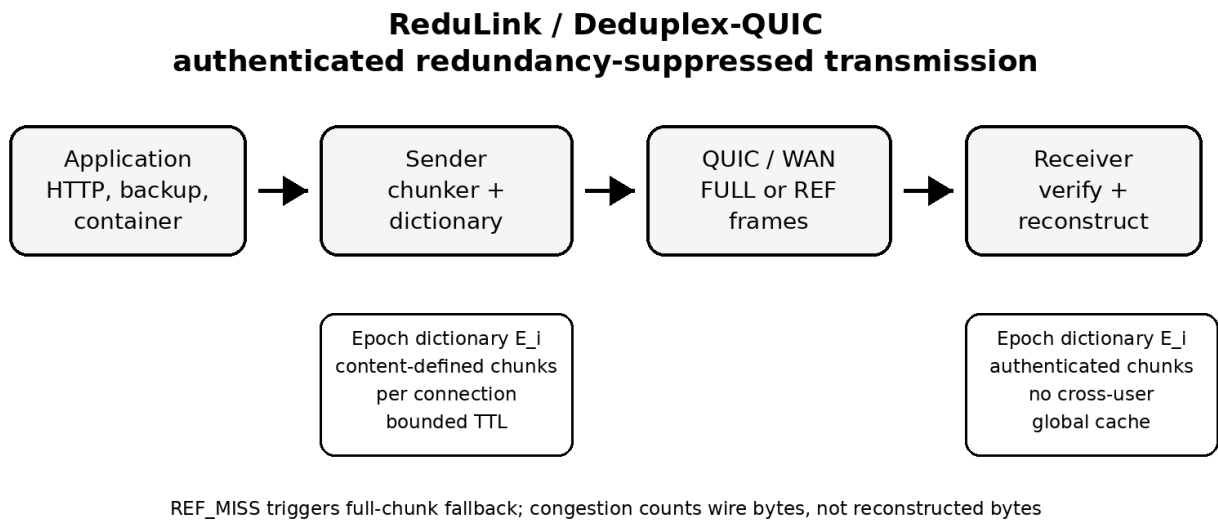


Figure 1. High-level ReduLink architecture.

4.1 Chunking

The sender divides outgoing payload into chunks. A practical deployment may use fixed-size chunks for CPU predictability, content-defined chunks for insertion resilience, or a hybrid strategy. The draft model assumes 8 KiB chunks and 48-byte references, but these values are parameters rather than constants.

4.2 Epoch dictionaries

An epoch dictionary is a temporary mapping from chunk identifiers to chunk bytes. A chunk identifier is derived from the chunk, the epoch, and the connection context. One possible construction is:

```
chunk_id = Truncate_128(HKDF(connection_secret, epoch_id || hash(chunk)))
```

This binding prevents references from one connection or epoch from being replayed into another context. The receiver stores only chunks received through authenticated FULL frames or otherwise confirmed through a secure dictionary establishment procedure.

4.3 Frame types

Frame or parameter	Purpose
redulink_parameters	Negotiates version, chunker, dictionary budget, expansion bound, scope, 0-RTT policy, and speculative-reference policy.
REDULINK_FULL	Carries original chunk bytes and metadata needed to validate and register the chunk in the receiver dictionary.
REDULINK_REF	Carries an authenticated reference to a chunk already present in the active receiver dictionary generation.
REDULINK_MISS	Indicates that the receiver cannot resolve a reference and requests semantic FULL repair for the same stream offset.
REDULINK_DICT_ACK	Optionally acknowledges receiver dictionary admission for conservative sender state.
epoch reset policy	Rotates dictionary state after key changes, memory pressure, excessive misses, or privacy policy changes.

Table 2. Proposed ReduLink frame types.

4.4 Core invariant

Invariant 1, authenticated reconstruction. A reference frame is deliverable if and only if the referenced chunk is present in the receiver dictionary, the epoch matches, the reference authentication succeeds, the reconstructed byte length remains within negotiated limits, and the reconstructed byte sequence occupies the committed stream position.

This invariant is the heart of the protocol. It distinguishes ReduLink from informal compression because it specifies exactly when reference substitution is safe.

5. Candidate Deduplex-QUIC profile

One candidate public-WAN profile is Deduplex-QUIC.

5.1 Negotiation

During connection setup, the endpoints exchange a transport parameter indicating ReduLink support. A conservative deployment would enable the feature only for selected origins, authenticated clients, enterprise VPNs, software-update channels, backup sessions, CDN transfers, or container registries.

```
redulink_transport_parameter = { version: 1, max_dictionary_bytes: 64 MiB, max_chunk_bytes: 64 KiB,
target_chunk_bytes: 8 KiB, max_reference_expansion: 64, max_pending_reconstructed_bytes: peer flow-
control bounded, max_ref_wait: local policy, privacy_mode: per_connection | per_origin,
fallback_required: true, zero_rtt_references: false }
```

5.2 Sending algorithm

```
for chunk in chunk_stream(payload):
    cid = chunk_id(chunk, epoch, dictionary_scope, stream_context)
    if receiver_state_allows_ref(cid) and safe_to_reference(chunk):
        send REDULINK_REF(epoch, stream_id, offset, cid, length, nonce, auth_tag)
    else:
        send REDULINK_FULL(epoch, stream_id, offset, cid, chunk, auth_tag)
        mark_receiver_candidate(cid)
```

5.3 Receiving algorithm

```
on REDULINK_FULL(epoch, stream_id, offset, cid, chunk, tag):
    verify tag, chunk id, length, epoch, and stream context
```

```

store dictionary[epoch][cid] = chunk
deliver chunk at offset

on REDULINK_REF(epoch, stream_id, offset, cid, length, nonce, tag):
    verify tag, epoch, length, offset, nonce, expansion bound, and dictionary presence
    if cid in dictionary[epoch] and length matches stored chunk:
        deliver dictionary[epoch][cid] at offset
    else:
        send REDULINK_MISS(epoch, stream_id, offset, cid)

```

5.4 Loss recovery

Loss recovery must not assume that sender and receiver dictionaries are perfectly synchronized. QUIC packet retransmission handles lost frame bytes, but once a REF arrives and fails dictionary resolution, resending the REF does not repair the semantic miss. The receiver blocks delivery and emits MISS; the sender repairs with a new REDULINK_FULL for the same stream id, reconstructed offset, original length, and chunk id.

5.5 Congestion semantics

ReduLink separates wire-byte accounting from reconstructed-byte accounting. Congestion control counts the bytes actually placed on the network. Application delivery counts the reconstructed bytes. This preserves fairness because a flow that sends 1 MiB of references consumes only 1 MiB of network capacity even if the receiver reconstructs 10 MiB of original payload.

6. Bandwidth model

Let R be the physical line rate, S the adjusted wire-byte saving, G the gross replaceable redundant share, q the reference-to-chunk size ratio, m the miss penalty, and o the control overhead. The model is:

```

G = raw_redundancy * replaceable_fraction
S = max(0, min(0.95, G * (1 - reference_bytes/chunk_bytes) - miss_rate * G - control_overhead))
T_effective = R / (1 - S)

```

The model is intentionally conservative. It does not assume that all repeated content is safely replaceable. It subtracts control overhead and reference misses. It also caps savings at 95 percent to prevent unrealistic infinite expansion.

6.1 Baseline 1 Gbit/s scenarios

Scenario	Raw redundancy	Replaceable share	Adjusted saving	Effective Gbit/s
Consumer HTTPS/media	0.15	0.30	0.037	1.04
Mixed enterprise SaaS	0.30	0.60	0.166	1.20
Corporate VPN/WAN	0.45	0.70	0.287	1.40
Log shipping	0.55	0.80	0.416	1.71
Software updates	0.65	0.75	0.467	1.88
Container layers	0.70	0.85	0.575	2.35
VM/backup replication	0.80	0.85	0.671	3.04
Compressed video	0.03	0.10	0.000	1.00
Encrypted random-like data	0.02	0.05	0.000	1.00

Table 3. Conservative 1 Gbit/s ReduLink model results.

The model indicates that ReduLink is not a universal Internet accelerator. It is weak for media-heavy consumer traffic and strong for workloads with repeated artifacts. This negative result is important because it prevents overclaiming and improves the credibility of the proposal.

	A	B	C	D	E	F	G	H	I	J
1	Deduplex / ReduLink Bandwidth-Savings Model for a 1 Gbit/s Line									
2	Purpose: estimate realistic wire-byte savings from authenticated redundancy-suppressed transmission. The model assumes a 1 Gbit/s physical link and computes reconstructed									
3	application payload throughput after reference overhead, misses, and control overhead.									
4										
5										
6	Traffic profile	Adjusted saving	Wire used for 1 GB	Effective throughput	Speed multiplier	Fit	Headline estimates			
7	Consumer HTTPS/media	0.037287187	985.81792	1.038731371	1.038731371 Low		Best public Internet case	10-30% saving	1.11-1.43 Gbit/s effective	
8	Mixed enterprise SaaS	0.166203594	853.80752	1.199333545	1.199333545 Medium		Corporate VPN/WAN	~29% adjusted	~1.41 Gbit/s effective	
9	Corporate VPN/WAN	0.286999325	730.1126912	1.402523216	1.402523216 Medium		VM/backup replication	~68% adjusted	~3.1 Gbit/s effective	
10	Log shipping	0.415783125	598.23808	1.711693111	1.711693111 Medium		Compressed video	~0% after overhead	No material gain	
11	Software updates	0.467347203	545.436464	1.877395568	1.877395568 High		Random encrypted payload	~0% after overhead	No material gain	
12	Container layers	0.574523675	435.6877568	2.350307035	2.350307035 High		Novelty sweet spot	Endpoint/QUIC/VPN	Not transparent routers	
13	VM/backup replication	0.670940975	336.9564416	3.038968465	3.038968465 High					
14	Compressed video	0	1024	1	1 Low					
15	Encrypted random-like data	0	1024	1	1 Low					

Figure 2. Draft dashboard from the ReduLink bandwidth model.

6.2 Interpretation

A 1 Gbit/s line with 30 percent adjusted saving can reconstruct about 1.43 Gbit/s of original payload. A 50 percent saving doubles effective reconstructed throughput to 2 Gbit/s. A 67 percent saving, plausible for carefully selected backup or VM replication workloads, reconstructs about 3 Gbit/s. These gains are workload-dependent and do not violate physical limits.

7. Security and privacy analysis

7.1 Threat model

The adversary may observe network traffic, drop packets, reorder packets, replay old packets, inject invalid frames, induce dictionary misses, and attempt to infer whether a receiver has seen particular content. The adversary is not assumed to break standard cryptographic primitives or compromise endpoint memory. In public-WAN mode, middleboxes are not trusted with plaintext.

Table 5a. Security properties, mechanisms, and artifact status.

Property	Claim	Required mechanism	Current artifact status
Integrity	Output equals input or fails closed.	FULL/REF authentication, chunk-id, offset, and length binding.	Reconstruction and mismatch tests; crypto not implemented.
Context binding	REF cannot replay across connection, epoch, stream, offset, origin, or scope.	Exporter-derived keys, epoch id, stream id, offset, dictionary id, nonce/replay window.	Specified, not implemented.
Dictionary safety	Only authenticated FULL or signed-manifest chunks are admitted.	Authenticated FULL, manifest commitment, admission and eviction policy.	Chunk-id checks modeled; manifest policy pending.
Expansion bound	Small REF cannot trigger unbounded work or delivery.	Per-frame, per-stream, and per-epoch reconstructed-byte caps.	Basic accounting modeled; QUIC flow-control enforcement pending.
Privacy scope	No private cross-user possession oracle in public mode.	Per-connection default; no global cross-user dictionary.	Policy specified; enforcement pending.

7.2 Reference forgery

A forged reference must fail authentication because the reference tag is derived from connection secrets, epoch identifiers, stream position, chunk identity, and length. An attacker cannot make the receiver deliver arbitrary dictionary bytes at an arbitrary stream offset without passing this check.

7.3 Replay

Epoch binding and stream-offset binding limit replay. A reference from one epoch is invalid in another epoch. A reference for one stream offset is invalid at another offset unless explicitly allowed by the reconstruction commitment.

7.4 Dictionary poisoning

The receiver stores only authenticated full chunks. A malicious sender can still send useless chunks, but that is no worse than sending useless payload. Receivers enforce dictionary limits, eviction policy, and expansion caps.

7.5 Expansion abuse

Reference-based reconstruction can create amplification inside the receiver. ReduLink therefore negotiates a maximum reference expansion ratio, per-epoch reconstructed-byte limits, and per-reference length limits. A reference cannot expand unboundedly.

7.6 Privacy side channels

Deduplication can leak information if a sender can determine whether another user or origin already has a chunk. ReduLink avoids this by defaulting to per-connection dictionaries. A more efficient per-origin mode can be allowed only when the privacy policy accepts the risk, such as within one authenticated enterprise VPN, one software-update channel, or one administrative domain. Global cross-user deduplication is excluded from public-WAN mode.

Like other deduplication systems, ReduLink can create a content-existence oracle when an adversary can choose payloads or references and observe wire size, timing, MISS count, fallback FULL size, DICT_ACK behavior, or application latency. Per-connection dictionaries remove the cross-user oracle in public-WAN mode, but they do not remove all leakage inside one connection, one origin, one tenant, or one administrative domain. Shared dictionaries are therefore limited to public artifacts, same-tenant deployments, or explicitly accepted policy domains.

Table 5b. Privacy modes and required controls.

Mode	Allowed dictionary scope	Leakage risk	Default?	Required controls
Public Internet	Per-connection only	Same-connection access-pattern leakage	Yes	No cross-user refs, 1-RTT only, bounded epochs
Public artifacts	Per-origin signed manifest	Artifact-version inference	Optional	Public-only content, manifest commitment, expiration
Enterprise VPN	Tenant/admin domain	Intra-tenant content-existence leakage	Optional	Tenant policy, quotas, audit, opt-out
Global cross-user	Any user	Private content-possession leakage	No	Out of scope

7.7 Simulator security scope and residual risks

The Python artifact is a reconstruction and accounting model, not a production QUIC implementation. It now verifies byte-exact reconstruction, REF-miss failure, FULL/REF length mismatches, miss-rate fallback behavior, and wire-byte accounting. It does not implement cryptographic authentication tags, replay windows, QUIC packetization, 0-RTT policy, production dictionary manifests, or cross-tenant privacy controls.

Shared dictionaries must remain per-connection by default. Per-origin dictionaries are appropriate only for public artifacts, a single administrative trust domain, or an explicit tenant/user policy. Chosen-chunk probing, dictionary poisoning, replayed references, expansion abuse, and MISS storms are treated as protocol threats with fail-closed requirements and bounded fallback, not as solved by the simulator alone.

8. Deployment models

8.1 Public Internet endpoint mode

In the public Internet case, ReduLink should be implemented by endpoints. Examples include browsers and servers, CDN edges and clients, software-update agents, backup tools, and container registries. The protocol is negotiated and encrypted end to end. Routers and middleboxes see ordinary encrypted traffic.

8.2 Enterprise VPN and branch WAN mode

Enterprise VPNs are an ideal early deployment target because both endpoints are controlled by the same organization. Repeated SaaS assets, file synchronization, logs, backups, and software packages can generate meaningful savings. Privacy policy is also easier because the administrative domain is unified.

8.3 CDN and software-update mode

CDNs and update services repeatedly deliver related artifacts to many clients. ReduLink can help if dictionaries are scoped safely, for example per client or per signed update channel. A software-update agent may retain a local dictionary of previously authenticated vendor chunks, allowing future updates to reference unchanged content.

8.4 Data-center and SmartNIC mode

In a trusted data-center fabric, ReduLink can run in host software, DPDK, eBPF, or SmartNIC firmware. The main research challenge is line-rate chunk detection without excessive CPU or memory use. SmartNICs can offload fingerprinting, dictionary lookup, and reference generation, but this increases implementation complexity.

9. Evaluation methodology

A credible evaluation should combine trace-driven simulation, prototype implementation, and security analysis.

9.1 Datasets

- Synthetic workloads with controlled redundancy from 0 to 90 percent.
- Container image versions and layer updates.
- Backup streams and virtual-machine image versions.
- Software-update packages with repeated libraries.
- Enterprise log streams with repeated templates.
- Negative-control datasets such as compressed video and random encrypted payloads.
- Git pack snapshots from a medium-sized public repository.
- Linux kernel release tarballs for version-to-version delta behavior.
- Ubuntu or Debian package metadata and OCI image layers as reproducible public artifacts.

9.2 Metrics

- Adjusted wire-byte saving.
- Effective reconstructed throughput.
- CPU cycles per byte.
- Memory consumed per active dictionary.
- Reference miss rate.
- Added latency.
- Goodput under packet loss.
- Fairness against non-ReduLink flows.
- Privacy leakage under adversarial probing.

9.3 Hypotheses

- H1: ReduLink provides negligible gain for already compressed or random encrypted payloads.
- H2: ReduLink provides 10 to 30 percent wire-byte savings for mixed enterprise traffic when both endpoints cooperate.
- H3: ReduLink provides 40 to 80 percent savings for selected backup, VM, container, and software-update workloads.
- H4: Reference-miss fallback preserves correctness under loss and dictionary mismatch.

- H5: Congestion fairness is preserved when congestion control accounts for wire bytes rather than reconstructed bytes.

9.4 Reproducible offline simulator and first results

To strengthen the manuscript beyond an analytic bandwidth model, this revision treats the repository as part of the evidence. The current artifact remains an endpoint-controlled redundancy-suppressed representation layer with epoch dictionaries, chunk identifiers, reference frames, bounded dictionary capacity, reference-miss fallback, and explicit wire-byte accounting. The repository now adds tests, baseline comparison commands, public-artifact hooks, local cost columns, and CSV-driven figure generation. The simulator is still not presented as a population estimate for the entire Internet; it is a controlled proof-of-concept showing when the protocol mechanism can and cannot create effective throughput gain.

Two chunking modes are evaluated. Content-defined chunking is used as the conservative generic-WAN mode because it can resynchronize after insertions and edits. Fixed record/page chunking is used as a controlled-mode model for artifacts that are naturally block- or record-aligned, such as VM pages, backup pages, container layers, update objects, and structured log frames. This distinction prevents the paper from overstating the method as a universal acceleration mechanism.

Table 4. Simulator parameters used in the proof-of-concept evaluation.

Parameter	Value	Purpose
Physical line rate	1 Gbit/s	Used to convert wire-byte savings into reconstructed payload throughput.
Average chunk size	4 KiB	Used for both CDC target size and fixed record/page mode.
Dictionary scope	session or warm origin dictionary	Cold mode learns only within a stream; warm mode models prior same-origin/stateful contexts.
Reference overhead	32 bytes	Current artifact REF metadata estimate; final QUIC wire encoding must recompute overhead from variants and tag policy.
Reference-miss rate	0.2%	Models occasional dictionary mismatch or repair fallback.
Dictionary capacity	64 MiB	LRU-bounded receiver dictionary.

Table 5. Selected ReduLink simulation results on a 1 Gbit/s physical line.

Workload	Mode	Saving	Effective payload	Interpretation
Consumer mixed	warm, cdc	20.70%	1.26 Gbit/s	Public Internet mixed traffic, endpoint dictionary
Enterprise SaaS	warm, cdc	42.43%	1.74 Gbit/s	Enterprise SaaS, content-defined chunks
Corporate VPN/WAN	warm, cdc	41.68%	1.72 Gbit/s	VPN/WAN generic stream
Log shipping	warm, fixed	19.80%	1.25 Gbit/s	Structured log frames, record-aligned mode
Software updates	warm, fixed	55.35%	2.24 Gbit/s	Versioned update object, origin dictionary
Container layers	cold, fixed	33.43%	1.50 Gbit/s	Container stream, repeated layers within session
Container layers	warm, fixed	59.80%	2.49 Gbit/s	Container stream, origin dictionary
VM backup replication	cold, fixed	66.27%	2.96 Gbit/s	Backup stream, repeated pages within session
VM backup replication	warm, fixed	88.58%	8.76 Gbit/s	Backup stream, prior snapshot dictionary
Compressed control	warm, cdc	0.00%	1.00 Gbit/s	Negative control
Encrypted random control	warm, cdc	0.00%	1.00 Gbit/s	Negative control

The selected results support three claims. First, the negative controls remain at the physical line rate, which is the expected behavior for compressed or random encrypted payloads. Second, generic endpoint-controlled WAN traffic benefits only when a safe same-origin or same-session dictionary is available. Third, controlled artifact flows can show substantial effective-throughput multiplication because the sender transmits compact authenticated references instead of repeated pages, layers, or update blocks.

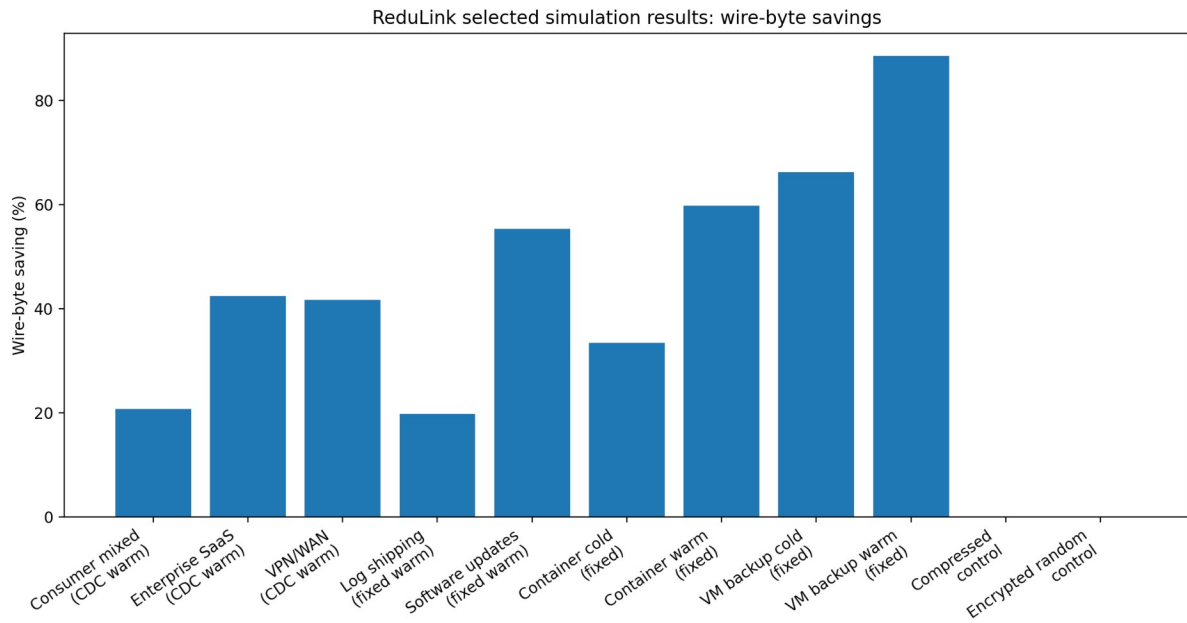


Figure 3. Selected ReduLink simulation results: wire-byte savings across workload families.

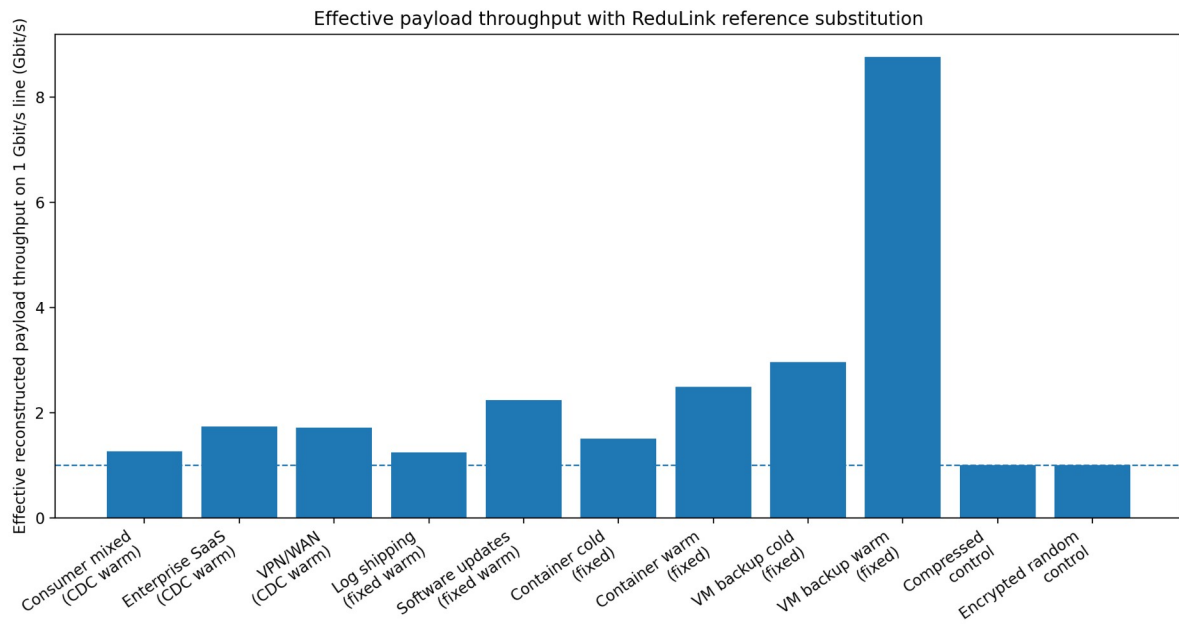


Figure 4. Effective reconstructed payload throughput on a 1 Gbit/s physical line.

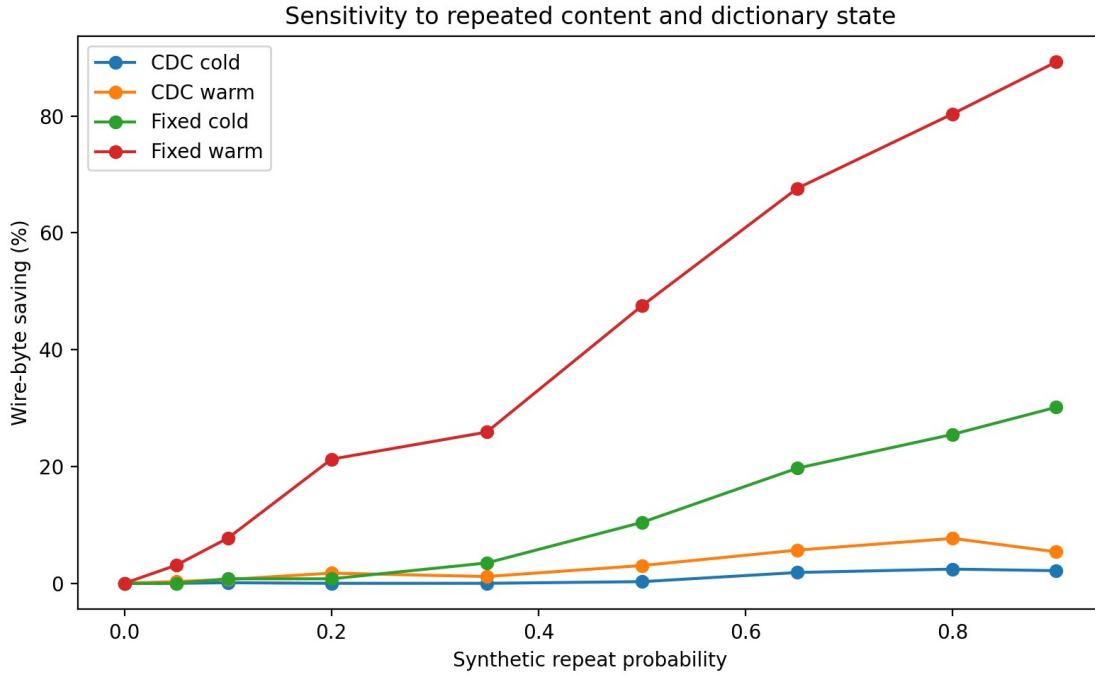


Figure 5. Sensitivity of wire-byte savings to repeated content and dictionary state.

These results also clarify the correct novelty claim. ReduLink is not a general replacement for caching, compression, or higher-rate Ethernet PHYs. Its strongest value is in cooperative endpoints that can safely maintain short-lived dictionaries before encryption or inside trusted endpoints. The most defensible deployment targets are VPNs, CDN-to-client delivery, software-update systems, container registries, VM replication, and backup streams.

The reproducibility package accompanying this draft includes the Python model, unit tests, GitHub Actions workflow, selected earlier artifact CSV, fetched public-corpora fixture, fixed-block reuse approximation, benchmark scripts with local cost columns, plot-generation script, generated figures, minimal socket prototype, threat model, and RFC-style protocol appendix. Larger validation should still run on frozen OCI layers, Linux kernel tarballs, git pack snapshots, package metadata, and VM/container snapshots before any production-scale claim.

9.5 Target-class evidence and baseline comparisons

The central evidence for v0.9 is the target-class suite in `results/target_class_suite.csv`. These deterministic generated fixtures are not production traces; they are controlled positive, weak, and negative cases that expose when warm dictionary references work. The table is deliberately honest: ReduLink loses on generated software-update and log-like cases, is modest on git-like data, is strong on VM/backup-like aligned pages, and remains near 1x on random and independent compressed negative controls.

The fixed-block reuse baseline is intentionally strong. It approximates delta-transfer behavior over exact block matches and is not wire-compatible rsync, but it is a serious comparator for update-like artifacts. It often beats ReduLink when object boundaries remain aligned, because exact block reuse avoids CDC cost and ReduLink pays FULL/REF framing overhead. ReduLink's narrower claim is not dominance over delta transfer; it is authenticated endpoint-controlled reference substitution with transport-aware dictionary, miss, privacy, and accounting semantics.

9.6 Public fixtures, tests, reproducible commands, and generated plots

The repository now includes tests for byte-exact FULL/REF reconstruction, random-data negative controls, warm-dictionary gains, safe REF-miss failure, public-manifest validation that skips cleanly until optional corpora are fetched, socket prototype byte-count accounting, and both fixed and content-defined chunkers. GitHub Actions runs

unit tests, public-fixture fetch, target-class generation, consistency checks, benchmark smoke tests, figure generation, evidence-table generation, and the socket demo.

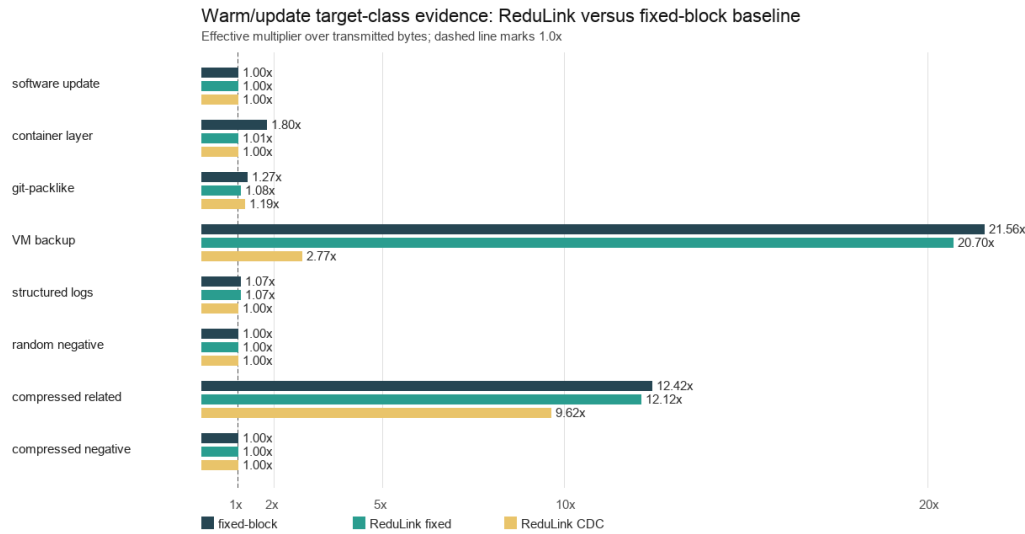
Benchmark reproduction is command-based: `python3 benchmarks/fetch_public_corpora.py` fetches pinned public pairs; `bash benchmarks/run_synthetic_suite.sh` emits `results/synthetic_suite.csv`; `bash benchmarks/run_public_artifacts.sh --manifest benchmarks/public_artifacts_manifest.csv` emits `results/public_artifact_suite.csv`; `bash benchmarks/run_target_class_suite.sh` emits `results/target_class_suite.csv`; `python3 benchmarks/check_generated_artifacts.py` checks generated target manifests and result labels; `python3 scripts/summarize_benchmark_evidence.py` regenerates `paper/evidence_tables.md`; and `python3 scripts/plot_warm_update_summary.py` regenerates the paper-facing Figure 6.

Table 6. v0.9 target-class evidence matrix from generated CSV outputs. The table includes weak and negative cases rather than only positive examples.

The current public fixture remains intentionally small: pinned text/version pairs from CPython, nginx, Redis, Linux documentation, and IETF QUIC RFCs. It is useful as a checksum-verifiable smoke fixture, not as production trace validation. Larger OCI layers, git packs, package repository metadata, VM/backup snapshots, and structured log archives remain required for stronger venue claims.

Target	Best compression	Fixed-block	ReduLink fixed	ReduLink CDC	Interpretation
software update	zstd-3 11.958x	1.000x	0.997x	0.999x	ReduLink loses; single-object compression dominates this generated update shape.
container layer	zstd-3 4.087x	1.799x	1.010x	0.999x	Weak reference identity; fixed-block baseline helps more than ReduLink.
git-packlike	zstd-3 18.080x	1.269x	1.082x	1.193x	Modest warm-dictionary gain, especially with CDC.
VM backup	zstd-3 9.949x	21.560x	20.701x	2.771x	Strong for aligned/page-like state; fixed-block and ReduLink fixed both benefit.
structured logs	gzip-6 12.397x	1.070x	1.067x	0.999x	Compression dominates; reference substitution is weak after overhead.
random negative	zstd-3 1.000x	1.000x	0.997x	0.998x	Correct no-gain random control.
compressed related	zstd-3 2.192x	12.417x	12.117x	9.623x	Diagnostic positive: related compressed streams retain reusable byte regions.
compressed negative	zstd-3 1.000x	1.000x	0.997x	0.998x	Correct no-gain compressed negative control.

Figure 6. v0.9 warm/update target-class evidence: fixed-block reuse versus ReduLink fixed and ReduLink CDC. Values near or below 1x are expected for weak and negative controls.



10. Prototype plan

A minimal prototype can be built in three stages.

10.1 Stage 1: offline simulator

The simulator reads byte streams, chunks them, calculates repeated chunks, replaces eligible repeats with references, and reports adjusted savings. The current toy model already implements the throughput equation. The next step is to replace synthetic redundancy assumptions with actual file and trace inputs.

The repository now also includes a minimal localhost socket prototype. In demo mode, a client sends modeled FULL/REF frames over TCP to a receiver, which reconstructs the update byte-exactly from transmitted frames and warm dictionary. The prototype validates endpoint cooperation and reconstruction only; it does not exercise QUIC packet protection, ACK/loss recovery, stream final-size handling, flow control, congestion fairness, 0-RTT, migration, or extension-frame parsing.

10.2 Stage 2: user-space tunnel

A user-space tunnel can implement ReduLink over UDP between two controlled hosts. This avoids modifying QUIC initially while testing dictionary synchronization, reference-miss recovery, memory behavior, and wire-byte savings.

10.3 Stage 3: QUIC extension or plugin

After the tunnel validates the core mechanism, the method can be implemented inside a QUIC library or as a pre-encryption application mapping. This stage evaluates deployability in HTTP-like transfers, software updates, and container delivery.

10.4 Stage 4: SmartNIC feasibility

The final stage estimates whether chunking and lookup can be offloaded to a SmartNIC. This stage should be treated as optional or future work unless a real programmable NIC environment is available.

11. Discussion

11.1 Why not normal compression?

Compression exploits statistical redundancy inside a local encoding window. ReduLink exploits repeated chunks across time, streams, files, sessions, or related artifacts. The two methods can coexist, but the order matters. Deduplication must occur before encryption and usually before compression if compression would destroy repeated byte identity. Alternatively, ReduLink can operate on application-level objects before they are compressed.

The benchmark suite therefore reports standalone compression baselines, a fixed-block reuse approximation, ReduLink fixed/CDC rows, and compression composition cases. This makes the distinction measurable: if zstd or gzip already removes repeated byte identity, ReduLink should not claim additional gain; if update-like artifacts preserve repeated chunk identity across history, ReduLink can reduce transmitted bytes even though congestion control still accounts only for wire bytes.

11.2 Why not middlebox WAN optimization?

Middlebox WAN optimization works well in controlled environments but conflicts with encrypted end-to-end traffic unless endpoints or administrators deliberately expose plaintext. ReduLink moves the redundancy-suppression decision to endpoints or trusted administrative boundaries, preserving compatibility with encrypted transports.

11.3 Why this is not merely old redundancy elimination

The novelty is not the observation that redundancy exists. The novelty is the protocolization of redundancy suppression for modern encrypted WANs: explicit authenticated reference frames, epoch dictionaries, reference-miss fallback, bounded expansion, congestion semantics, and privacy-preserving deployment rules. These elements are necessary for a deployable Internet-era design.

11.4 Scope of claims

ReduLink is not a faster link, a universal accelerator, a compression replacement, a delta-transfer replacement, or a completed QUIC implementation. The contribution is a scoped protocol model for authenticated reference substitution in cooperative endpoints, with explicit reconstruction invariants, miss repair, dictionary privacy boundaries, expansion limits, and reproducible evidence showing both useful and weak target classes.

12. Limitations

- The current evaluation separates analytic modeling, deterministic target-class fixtures, small frozen public-corpora fixtures, and a TCP endpoint prototype. It still requires larger public corpora and real QUIC implementation experiments before production-scale claims.
- The method provides little benefit for compressed media, random encrypted data, and unique one-time payloads.
- Endpoint integration is required for encrypted public-WAN traffic.
- Content-defined chunking may be CPU-intensive at high line rates without optimization or offload.
- Privacy-safe dictionary sharing limits the maximum savings compared with unsafe global deduplication.
- The design requires careful interaction with application compression, caching, and content encoding.

13. Conclusion

ReduLink reframes network acceleration from physical-speed escalation to effective reconstructed payload throughput. A 1 Gbit/s link cannot physically transmit more than its line rate, but a receiver can reconstruct more than 1 Gbit/s of original payload when repeated bytes are replaced by authenticated references. The proposed protocol makes this idea compatible with modern encrypted WANs by using endpoint negotiation, epoch-scoped dictionaries,

authenticated references, bounded expansion, reference-miss fallback, and congestion accounting based on wire bytes. The expected gains are workload-dependent and carry CPU, memory, and privacy costs. The resulting research contribution is not a faster Ethernet PHY, but a security-scoped transport-layer or link-adjacent primitive for effective bandwidth expansion under redundancy.

References

- [1] N. T. Spring and D. Wetherall, "A protocol-independent technique for eliminating redundant network traffic," ACM SIGCOMM, 2000.
- [2] A. Anand, V. Sekar, and A. Akella, "SmartRE: An architecture for coordinated network-wide redundancy elimination," ACM SIGCOMM, 2009.
- [3] A. Anand et al., "Redundancy in network traffic: Findings and implications," ACM SIGMETRICS, 2009.
- [4] B. Aggarwal et al., "EndRE: An end-system redundancy elimination service for enterprises," USENIX NSDI, 2010.
- [5] A. Muthitacharoen, B. Chen, and D. Mazières, "A low-bandwidth network file system," ACM SOSP, 2001.
- [6] J. Iyengar and M. Thomson, "QUIC: A UDP-based multiplexed and secure transport," RFC 9000, IETF, 2021. Available: <https://datatracker.ietf.org/doc/rfc9000/>
- [7] M. Thomson and S. Turner, "Using TLS to secure QUIC," RFC 9001, IETF, 2021.
- [8] M. Kühlewind and B. Trammell, "Applicability of the QUIC transport protocol," RFC 9308, IETF, 2022.
- [9] B. Trammell et al., "Manageability of the QUIC transport protocol," RFC 9312, IETF, 2022. Available: <https://datatracker.ietf.org/doc/rfc9312/>
- [10] Y. Hu et al., "The design of fast content-defined chunking for data deduplication," IEEE Transactions on Parallel and Distributed Systems, 2020.
- [11] M. Gregoriadis, L. Balduf, B. Scheuermann, and J. Pouwelse, "A thorough investigation of content-defined chunking algorithms for data deduplication," arXiv, 2024.
- [12] B. Alexeev, C. Percival, and Y. X. Zhang, "Chunking attacks on file backup services using content-defined chunking," arXiv, 2025.
- [13] IEEE 802.3 Ethernet Working Group, "IEEE 802.3 Ethernet Working Group active projects," accessed June 2026. Available: <https://www.ieee802.org/3/>
- [14] M. Nguyen, "ReduLink repository and reproducibility package," GitHub repository, version 0.9, 2026. Available: <https://github.com/pinkysworld/redulink-deduplex-quic>
- [15] D. Harnik, B. Pinkas, and A. Shulman-Peleg, "Side channels in cloud services: Deduplication in cloud storage," IEEE Security & Privacy, 2010.
- [16] M. Bellare, S. Keelveedhi, and T. Ristenpart, "DupLESS: Server-aided encryption for deduplicated storage," USENIX Security, 2013.

Appendix A: Reproducibility package contents

The v0.9 reproducibility package contains the runnable model, compatibility wrapper, tests, GitHub Actions workflow, synthetic CSV, public-corpora manifest and result CSV, target-class manifest and result CSV, target-class summary CSV, fixed-block reuse approximation, benchmark scripts with local wall-clock/RSS cost columns, plot-generation scripts, regenerated figures, minimal socket prototype, threat model, and RFC-style protocol appendix.

```
python3 -m unittest discover -s tests
```

```
bash benchmarks/run_synthetic_suite.sh
```

```
python3 prototypes/redulink_socket_prototype.py demo
```

```
python3 benchmarks/fetch_public_corpora.py
```

```
bash benchmarks/run_public_artifacts.sh --manifest benchmarks/public_artifacts_manifest.csv
```

```
python3 scripts/plot_results.py results/synthetic_suite.csv --output-dir figures
```

The CDC run represents conservative generic stream chunking, while the fixed run represents record/page-aligned controlled artifact flows. Public-artifact runs should report artifact source, version, retrieval date, chunker, chunk size, dictionary limit, miss-rate assumption, and whether the run is cold or warm/update-like.

Appendix B: Example frame format

The full repository appendix expands this sketch into negotiation parameters, frame types, sender and receiver state machines, DICT_ACK semantics, warm dictionary manifests, reference-miss repair, security requirements, congestion and flow-control accounting, and failure cases.

```
REDULINK_REF {
    frame_type: extension-assigned varint,
    epoch_id: uint64,
    stream_id: varint,
    stream_offset: varint,
    original_length: varint,
    chunk_id: 128 bits,
    reference_nonce: 96 bits,
    auth_tag_or_commitment: 128 bits
}
```

Appendix C: Conservative claim language

Recommended claim: ReduLink increases effective reconstructed payload throughput for redundant traffic without increasing physical line rate or weakening transport security.

Avoided claim: ReduLink makes Ethernet physically faster than its negotiated line rate.